Spirit Gate - Guia das fixtures do Passo Relâmpago

TASK-006.1. Consulta ao código local em 18/10/2026.

Este guia reúne as assinaturas reais para criar uma parede sólida, montar o espaço de colisão e executar o dash seletivo até o encerramento. O exemplo usa as fixtures existentes e executa somente o domínio, sem Swing, rede ou banco de dados.

O teste JUnit devePreservarBloqueioDeParedeNoDashSeletivo continua reservado ao usuário. O exemplo demonstra a montagem e a execução; não inclui @Test, asserções de posição nem o resultado esperado da colisão com a parede. Nenhum arquivo de teste foi alterado nesta documentação.

1. Localização e acesso às fixtures

Arquivo: server/src/test/java/com/projetok/domain/skill/LightningStepTestFixtures.java.

A classe LightningStepTestFixtures e seus membros são package-private. Seu código precisa estar no pacote com.projetok.domain.skill e ter os arquivos de teste compilados no classpath. As fixtures não fazem parte da API de produção.

Assinaturas reais:

```
static final float RADIUS = 18f;
static Player player(float x, float y);
static CollisionService.Space emptySpace();
static LightningStepService service();
static LightningStepService selectiveService();
```

player(x,y) cria um jogador humano pela PlayerFactory real, com 100 de vida e 100 de vigor. RADIUS é o raio de colisão do jogador, em unidades do mundo. Não é o diâmetro.

emptySpace() cria uma arena de 960 por 640, sem obstáculos, corpos ou índice espacial. Para incluir uma parede, crie outro Space com a lista de obstáculos.

service() conserva a política SOLID. Para a TASK-006.1, use selectiveService(): ela monta MovementService e LightningStepService compartilhando a mesma instância de CollisionService e seleciona PHASE_THROUGH_ACTORS_AND_SOFT.

Importações para usar as fixtures no mesmo pacote:

```
package com.projetok.domain.skill;

import static com.projetok.domain.skill.LightningStepTestFixtures.RADIUS;
import static com.projetok.domain.skill.LightningStepTestFixtures.player;
import static com.projetok.domain.skill.LightningStepTestFixtures.selectiveService;
```

2. Criar uma parede sólida

Arquivo: server/src/main/java/com/projetok/domain/movement/CollisionService.java.

O construtor de quatro argumentos mantém a categoria sólida por padrão:

```
public Obstacle(float x, float y, float width, float height)
```

O construtor canônico do record permite informar a categoria explicitamente:

```
public Obstacle(float x, float y, float width, float height,
                CollisionCategory category)
```

x e y indicam o canto superior esquerdo do retângulo; width e height são suas dimensões. Coordenadas e extremidades precisam ser finitas; as dimensões devem ser positivas. Para retângulos, são aceitas SOFT_OBSTACLE e SOLID_OBSTACLE.

As duas formas abaixo criam a mesma parede sólida; escolha uma delas:

```
var parede = new CollisionService.Obstacle(200f, 60f, 20f, 160f);

// Forma alternativa com categoria explícita:
var paredeExplicita = new CollisionService.Obstacle(
        200f, 60f, 20f, 160f, CollisionCategory.SOLID_OBSTACLE);
```

A política seletiva ignora atores e obstáculos leves. Uma parede SOLID_OBSTACLE continua bloqueante. WORLD_BOUNDARY representa as bordas; elas já são derivadas das dimensões de Space.

3. Criar o espaço de colisão

Construtores reais de Space e Body, ambos records internos de CollisionService:

```
public Space(float width, float height,
             List<Obstacle> obstacles, List<Body> bodies)

public Space(float width, float height,
             List<Obstacle> obstacles, List<Body> bodies,
             SpatialBodyIndex index)

public Body(GameEntity entity, float radius)
```

O construtor de quatro argumentos não usa índice espacial. Ele é suficiente para um cenário pequeno de teste. Space copia as listas; as posições e a vida dos atores são consultadas ao vivo. O próprio jogador é ignorado nas consultas de colisão contra seu corpo.

```
var jogador = player(100f, 100f);
var parede = new CollisionService.Obstacle(200f, 60f, 20f, 160f);
var corpoJogador = new CollisionService.Body(jogador, RADIUS);

var espaco = new CollisionService.Space(
        960f, 640f, List.of(parede), List.of(corpoJogador));
```

Neste exemplo, o raio do jogador pertence a Body e também é informado nas chamadas do serviço. A parede é um retângulo: não recebe raio. Não reutilize emptySpace() se a intenção for testar a parede, pois ele não contém esse obstáculo.

Se optar pelo construtor com SpatialBodyIndex, construa o índice com as mesmas instâncias de Body da lista do Space. Movimentos feitos pela física atualizam o índice; uma alteração direta externa de posição exige index.update(entity). O exemplo deste guia usa a versão sem índice para reduzir a montagem.

4. Contrato real do dash

Arquivo do comando: server/src/main/java/com/projetok/domain/skill/SkillCastCommand.java.

```
public record SkillCastCommand(String skillId, float targetX, float targetY)
```

Arquivo do serviço:
server/src/main/java/com/projetok/domain/skill/LightningStepService.java.

```
public LightningStepService(MovementService movement,
                            StaminaService stamina,
                            CollisionService collisions)

public LightningStepService(MovementService movement,
                            StaminaService stamina,
                            CollisionService collisions,
                            CollisionPolicy policy)

public Status cast(Player player, SkillCastCommand command,
                   float radius, CollisionService.Space space)

public Step update(Player player, float dt, float radius,
                   CollisionService.Space space)

public boolean active()
public boolean awaitingLanding()
public float remaining()
public float cooldown()
```

O construtor antigo usa SOLID; selectiveService() usa o novo construtor com a política seletiva. Status e Step são tipos internos de LightningStepService. Step contém from(), to() e path(), além de moved(); path() é uma lista imutável dos vértices realmente percorridos.

cast valida e inicia o estado. O retorno STARTED confirma a aceitação; a chamada ainda não desloca o jogador. O comando usa LightningStepService.ID, cujo valor é "lightning_step". O ponto alvo deve ser finito, dentro da arena e diferente da posição do jogador. Ele fixa a direção, e não um destino obrigatório.

update avança o tempo e move usando colisão contínua. dt é dado em segundos: deve ser finito e não negativo. A velocidade é 600 px/s; o orçamento direcional é de até 150 px durante 0,25 s. O custo é 35 e o cooldown é 4 s. Não chame cast a cada atualização: a ativação é única, seguida das chamadas de update.

O último avanço usa somente o tempo restante, mesmo se dt for maior. Uma parede pode bloquear o deslocamento antes do fim, mas o relógio do dash continua até encerrar. O cooldown não precisa chegar a zero para concluir o dash: ele continua após a duração de movimento.

O cooldown avança pelo dt completo de cada update; somente o deslocamento é limitado ao tempo restante. O exemplo mantém o passo fixo de 1/30 s, como a arena. Para controlar exatamente o tempo de uma atualização final isolada, pode-se limitar seu delta a Math.min(dt, service.remaining()) enquanto active() for verdadeiro.

active() indica tempo de dash restante. awaitingLanding() indica espera por uma saída normalmente livre quando atores móveis ocupam toda a trajetória. Nessa situação, active() pode ser falso e a saída ainda estar pendente. O loop de exemplo é limitado e considera ambos os estados, evitando uma espera infinita. Um cenário estático com apenas uma parede sólida não depende de liberação de atores.

5. Exemplo completo de montagem e execução

Este exemplo é uma classe executável de demonstração, não um teste JUnit. O pacote é necessário para acessar as fixtures. Não o coloque em src/main/java: ele depende de classes de src/test/java. Para aproveitar no seu teste, use apenas a montagem e as chamadas que precisar; as asserções continuam a seu cargo.

```
package com.projetok.domain.skill;
```

```
import com.projetok.domain.movement.CollisionService;
import java.util.List;

import static com.projetok.domain.skill.LightningStepTestFixtures.RADIUS;
import static com.projetok.domain.skill.LightningStepTestFixtures.player;
import static com.projetok.domain.skill.LightningStepTestFixtures.selectiveService;

/** Demonstra as fixtures sem implementar o teste reservado. */
public final class ExemploFixturesDash {
    /** Executa a montagem e os passos em uma única thread. */
    public static void main(String[] args) {
        var jogador = player(100f, 100f);
        var parede = new CollisionService.Obstacle(
                200f, 60f, 20f, 160f);
        var corpo = new CollisionService.Body(jogador, RADIUS);
        var espaco = new CollisionService.Space(
                960f, 640f, List.of(parede), List.of(corpo));
        var service = selectiveService();
        var comando = new SkillCastCommand(
                LightningStepService.ID, 800f, 100f);

        var resultado = service.cast(jogador, comando, RADIUS, espaco);
        if (resultado != LightningStepService.Status.STARTED) {
            throw new IllegalStateException("Dash recusado: " + resultado);
        }

        float dt = 1f / 30f;
        int ticks = 0;
        int limite = 30;
        while ((service.active() || service.awaitingLanding()) && ticks < limite) {
            service.update(jogador, dt, RADIUS, espaco);
            ticks++;
        }

        if (service.active() || service.awaitingLanding()) {
            throw new IllegalStateException("Simulacao terminou com dash pendente");
        }
        System.out.println("Dash concluido em " + ticks + " atualizacoes.");
    }
}
```

O guard de 30 atualizações é uma proteção da demonstração; não é a duração da habilidade. A duração vem do serviço. Não há Thread.sleep: cada update representa um passo da simulação. O exemplo não aciona regeneração de vigor, IA ou outros serviços de DebugWorld.

Para inspecionar o estado depois das chamadas, existem jogador.getPosition().x()/y(), service.remaining(), service.cooldown(), service.active() e service.awaitingLanding(). As asserções sobre parede e raio devem ser escritas por você no arquivo server/src/test/java/com/projetok/domain/skill/LightningStepSelectiveCollisionTest.java.

6. Verificação e reprodução

O exemplo do bloco anterior foi extraído deste documento, compilado com javac --release 17 usando target/classes e target/test-classes e executado. Ambos os comandos terminaram com código de saída 0. Saída real: "Dash concluido em 8 atualizacoes."

A compilação e a execução são independentes da janela. Arquivos temporários de demonstração ficam em server/target, fora dos fontes e dos testes do projeto. Não foi criado um teste JUnit para o cenário reservado nem executada novamente a suíte Maven nesta alteração documental.

Para reproduzir, salve a classe do exemplo em target/guia-fixtures-src/com/projetok/domain/skill/ExemploFixturesDash.java. Com as classes do projeto e das

fixtures já compiladas, estes foram os comandos executados em PowerShell, na pasta server:

```
javac --release 17 -encoding UTF-8 `
  -cp 'target/classes;target/test-classes' `
  -d target/guia-fixtures `
  target/guia-fixtures-src/com/projetok/domain/skill/ExemploFixturesDash.java

java -cp 'target/classes;target/test-classes;target/guia-fixtures' `
  com.projetok.domain.skill.ExemploFixturesDash
```

Para compilar as fixtures quando target não estiver disponível, execute mvn test-compile na pasta server. Isso prepara as classes de produção e teste. Para validar sua contribuição JUnit posteriormente, execute mvn clean test na mesma pasta.

O PDF é regenerado na pasta Projeto-K com python -B tools/gerar_pdf_guia_fixtures_dash.py. Este Markdown é a fonte editável do documento. A documentação não modifica a implementação de colisão, do dash ou das fixtures.